Sentence Memorability: Data Visualization

```r
library(readr)
library(dplyr)
library(purrr)
library(stringr)
library(tidyr)
library(ggplot2)

# ---------------------------------------------------
# File paths
# ---------------------------------------------------
# First try local files in the same folder as the Rmd.
# If not found, fall back to /mnt/data paths.
zip_path <- "/Users/hanvitha/Downloads/Sentence Memorability.zip"


brief_path <- "/Users/hanvitha/Downloads/Sentence Memorability (1).txt"


if (!file.exists(zip_path)) {
  stop("Could not find 'Sentence Memorability.zip'. Place it in the same folder as this Rmd.")
}

# ---------------------------------------------------
# Unzip logs
# ---------------------------------------------------
unzip_dir <- file.path(tempdir(), "sentence_memorability_logs")
if (dir.exists(unzip_dir)) unlink(unzip_dir, recursive = TRUE)
dir.create(unzip_dir, recursive = TRUE)

unzip(zip_path, exdir = unzip_dir)

log_dir <- file.path(unzip_dir, "NewLogsAnonymized")
if (!dir.exists(log_dir)) {
  stop("Folder 'NewLogsAnonymized' was not found inside the ZIP.")
}

log_files <- list.files(log_dir, pattern = "\\.log$", full.names = TRUE)
if (length(log_files) == 0) {
  stop("No .log files were found inside 'NewLogsAnonymized'.")
}

# ---------------------------------------------------
# Read logs
# ---------------------------------------------------
read_one_log <- function(path) {
  read_csv(
```

```r
    path,
    show_col_types = FALSE,
    na = c("N/A", "", "NA")
  ) %>%
    mutate(file = basename(path)) %>%
    arrange(Timestamp)
}

raw_logs <- map_dfr(log_files, read_one_log)

# ---------------------------------------------------
# Keep only experimental events (exclude practice)
# ---------------------------------------------------
exp_logs <- raw_logs %>%
  filter(!str_detect(Event, "^Practice"))

# ---------------------------------------------------
# Parse event stream into trial-level data
# ---------------------------------------------------
parse_trials <- function(df) {
  df <- df %>% arrange(Timestamp)

  started <- FALSE
  block <- 0L
  out <- list()
  current <- NULL
  seen <- list()

  finalize_current <- function(cur, out_list) {
    if (!is.null(cur)) out_list[[length(out_list) + 1]] <- cur
    out_list
  }

  for (i in seq_len(nrow(df))) {
    row <- df[i, ]
    event <- row$Event

    if (identical(event, "Sentence shown")) {
      if (!started) {
        started <- TRUE
        block <- 1L
      }

      out <- finalize_current(current, out)

      stim <- row$Stimulus
      m <- str_match(stim, "^([A-Z]+)_([0-9]+)_([AP])$")

      if (all(is.na(m))) next

      cond  <- m[, 2]
      sid   <- as.integer(m[, 3])
      voice <- m[, 4]
```

```r
    base  <- paste(cond, sid, sep = "_")

    repeat_flag <- isTRUE(tolower(as.character(row$isRepeat)) == "true")
    prev_voices <- seen[[base]]

    rep_type <- "new"
    if (repeat_flag) {
      if (!is.null(prev_voices) && voice %in% prev_voices) {
        rep_type <- "exact"
      } else if (!is.null(prev_voices) && length(prev_voices) > 0) {
        rep_type <- "similar"
      } else {
        rep_type <- "repeat_no_prev"
      }
    }

    seen[[base]] <- c(prev_voices, voice)

    current <- tibble(
      participant_ID = row$participant_ID,
      block = block,
      stimulus = stim,
      cond = cond,
      sid = sid,
      voice = voice,
      isTarget = isTRUE(tolower(as.character(row$isTarget)) == "true"),
      isRepeat = repeat_flag,
      rep_type = rep_type,
      ir_pressed = FALSE,
      ir_rt = NA_real_,
      wr_pressed = FALSE,
      wr_rt = NA_real_,
      wr_accuracy = NA_real_
    )

  } else if (identical(event, "Rest Phase started")) {
    out <- finalize_current(current, out)
    current <- NULL
    if (started) {
      block <- block + 1L
      seen <- list()
    }

  } else {
    if (is.null(current)) next

    if (str_detect(event, "IR pressed")) {
      current$ir_pressed <- TRUE
      current$ir_rt <- suppressWarnings(as.numeric(row$Reaction_time_IR))
    }

    if (identical(event, "WR pressed")) {
      current$wr_pressed <- TRUE
```

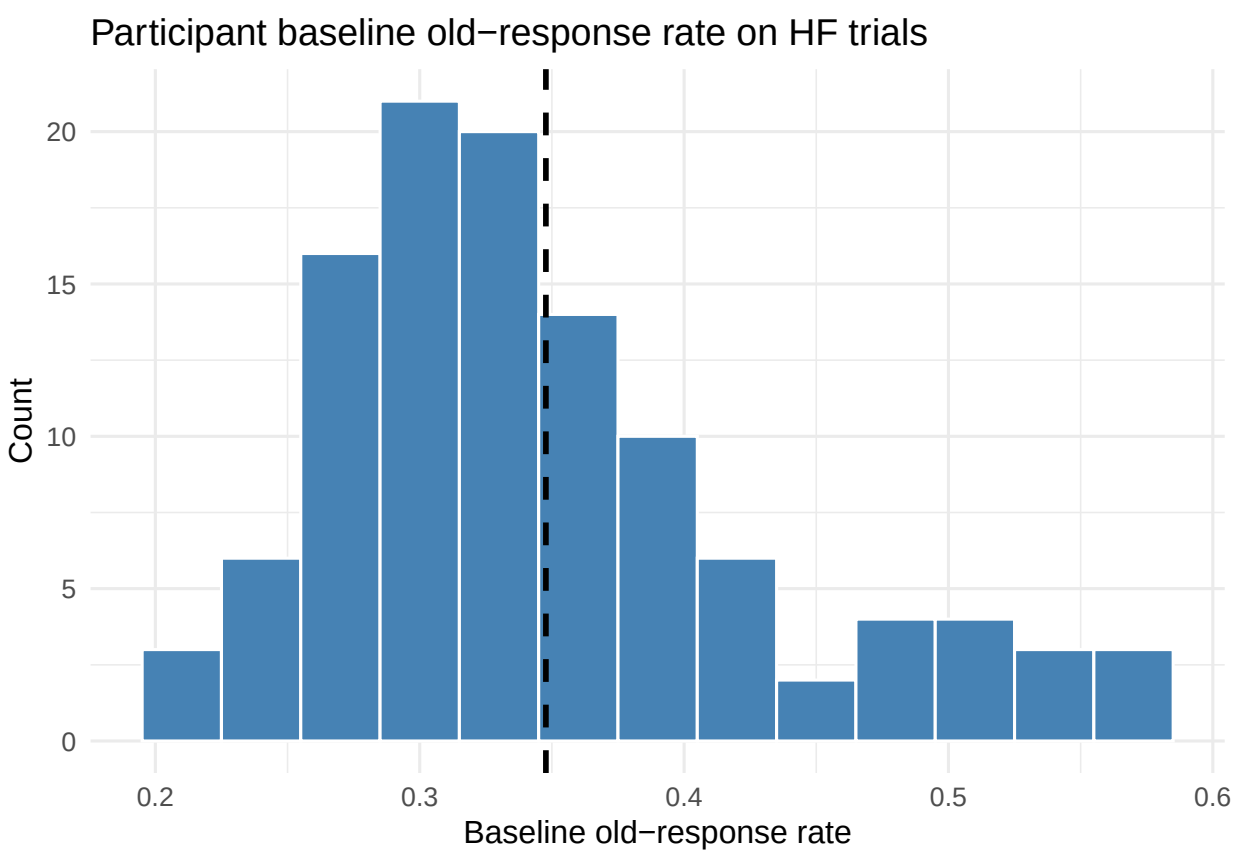
```r
        current$wr_rt <- suppressWarnings(as.numeric(row$Reaction_time_WR))
        current$wr_accuracy <- suppressWarnings(as.numeric(row$`Accuracy WR`))
      }
    }
  }

  out <- finalize_current(current, out)

  if (length(out) == 0) {
    return(tibble())
  }

  bind_rows(out)
}

trial_data <- exp_logs %>%
  group_by(participant_ID) %>%
  group_modify(~ parse_trials(.x)) %>%
  ungroup()

# ---------------------------------------------------
# Validation pass / fail per block
# ---------------------------------------------------
assign_blocks_for_validation <- function(df) {
  df <- df %>% arrange(Timestamp)

  started <- FALSE
  block <- 0L
  block_id <- rep(NA_integer_, nrow(df))

  for (i in seq_len(nrow(df))) {
    if (!started && df$Event[i] == "Sentence shown") {
      started <- TRUE
      block <- 1L
    }

    if (started) block_id[i] <- block

    if (started && df$Event[i] == "Rest Phase started") {
      block <- block + 1L
    }
  }

  df$block <- block_id
  df
}

validation_df <- exp_logs %>%
  group_by(participant_ID) %>%
  group_modify(~ assign_blocks_for_validation(.x)) %>%
  ungroup() %>%
  filter(!is.na(block)) %>%
  group_by(participant_ID, block) %>%
```

```r
  summarise(
    correct = sum(Event == "Validation IR pressed", na.rm = TRUE),
    wrong   = sum(Event == "Validation Wrong IR pressed", na.rm = TRUE),
    missed  = sum(Event == "Validation Missed", na.rm = TRUE),
    pass    = correct > (wrong / 2 + missed),
    .groups = "drop"
  )

valid_trials <- trial_data %>%
  inner_join(
    validation_df %>% filter(pass) %>% select(participant_ID, block),
    by = c("participant_ID", "block")
  )

# ---------------------------------------------------
# Condition renaming
# ---------------------------------------------------
valid_trials <- valid_trials %>%
  mutate(
    stype = case_when(
      cond == "HH"  ~ "HH",
      cond == "HVL" ~ "HL",
      cond == "LVH" ~ "LH",
      cond == "LVL" ~ "LL",
      TRUE ~ cond
    )
  )

# ---------------------------------------------------
# Participant-level summaries
# ---------------------------------------------------
baseline_df <- valid_trials %>%
  filter(stype == "HF") %>%
  group_by(participant_ID) %>%
  summarise(
    baseline_old_rate = mean(ir_pressed, na.rm = TRUE),
    .groups = "drop"
  )

hit_df <- valid_trials %>%
  filter(stype %in% c("HH", "HL", "LH", "LL"), isTarget, isRepeat) %>%
  group_by(participant_ID, stype, voice) %>%
  summarise(
    hit_rate = mean(ir_pressed, na.rm = TRUE),
    .groups = "drop"
  )

score_df <- hit_df %>%
  left_join(baseline_df, by = "participant_ID") %>%
  mutate(corrected_score = hit_rate - baseline_old_rate)

rt_df <- valid_trials %>%
  filter(stype %in% c("HH", "HL", "LH", "LL"), isTarget, isRepeat, ir_pressed) %>%
```

```r
  group_by(participant_ID, stype, voice) %>%
  summarise(
    ir_rt_mean = mean(ir_rt, na.rm = TRUE),
    .groups = "drop"
  )

wr_df <- valid_trials %>%
  filter(stype %in% c("HH", "HL", "LH", "LL"), isTarget, isRepeat, ir_pressed, wr_pressed) %>%
  group_by(participant_ID, stype, voice, rep_type) %>%
  summarise(
    wr_acc = mean(wr_accuracy, na.rm = TRUE),
    .groups = "drop"
  )
```

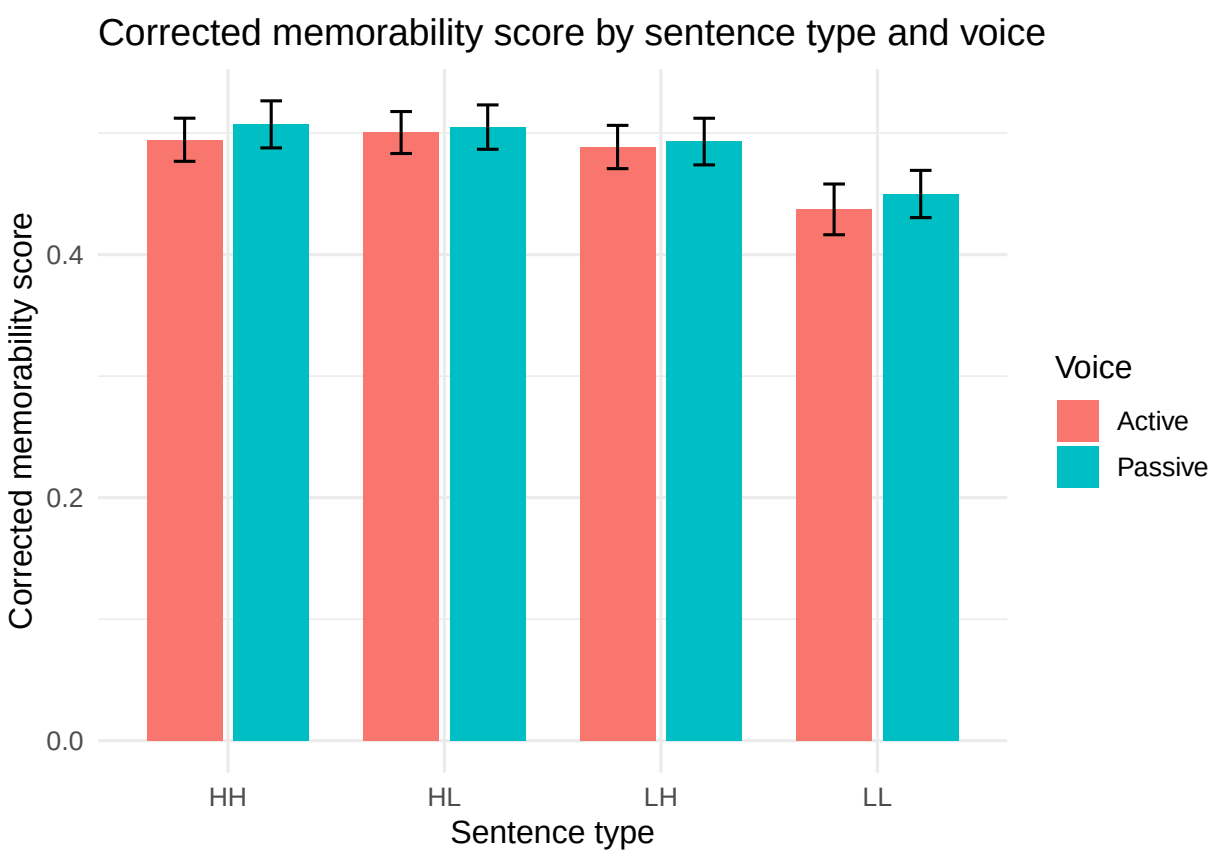
Plot 1: baseline old-response rate

```r
ggplot(baseline_df, aes(x = baseline_old_rate)) +
  geom_histogram(binwidth = 0.03, fill = "steelblue", color = "white") +
  geom_vline(xintercept = mean(baseline_df$baseline_old_rate, na.rm = TRUE),
             linetype = "dashed", linewidth = 1) +
  labs(
    title = "Participant baseline old-response rate on HF trials",
    x = "Baseline old-response rate",
    y = "Count"
  ) +
  theme_minimal(base_size = 12)
```



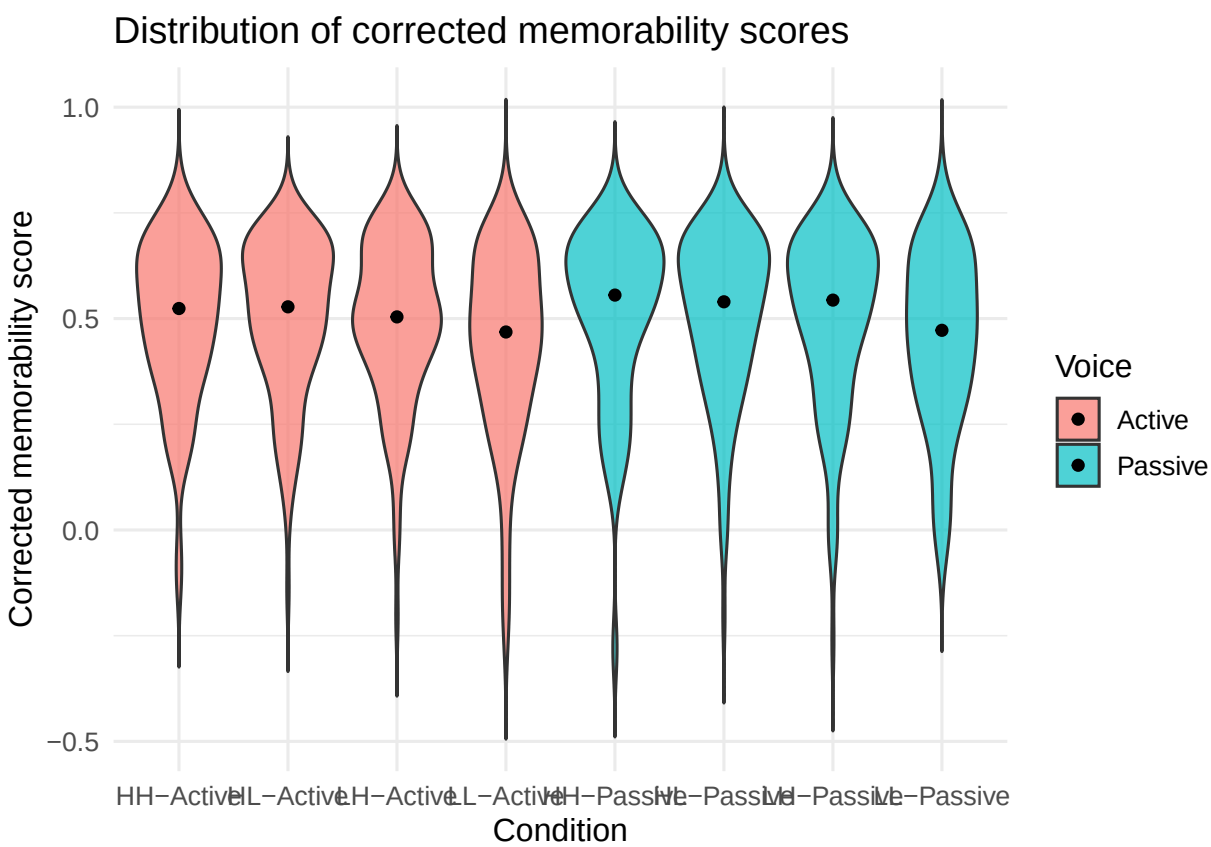
Plot 2: corrected memorability by sentence type and voice

```
score_plot_df <- score_df %>%
  mutate(
    stype = factor(stype, levels = c("HH", "HL", "LH", "LL")),
    voice = factor(voice, levels = c("A", "P"), labels = c("Active", "Passive"))
  )

ggplot(score_plot_df, aes(x = stype, y = corrected_score, fill = voice)) +
  stat_summary(fun = mean, geom = "bar",
               position = position_dodge(width = 0.8), width = 0.7) +
  stat_summary(
    fun.data = mean_se,
    geom = "errorbar",
    position = position_dodge(width = 0.8),
    width = 0.2
  ) +
  labs(
    title = "Corrected memorability score by sentence type and voice",
    x = "Sentence type",
    y = "Corrected memorability score",
    fill = "Voice"
  ) +
  theme_minimal(base_size = 12)
```



Plot 3: distribution of corrected memorability

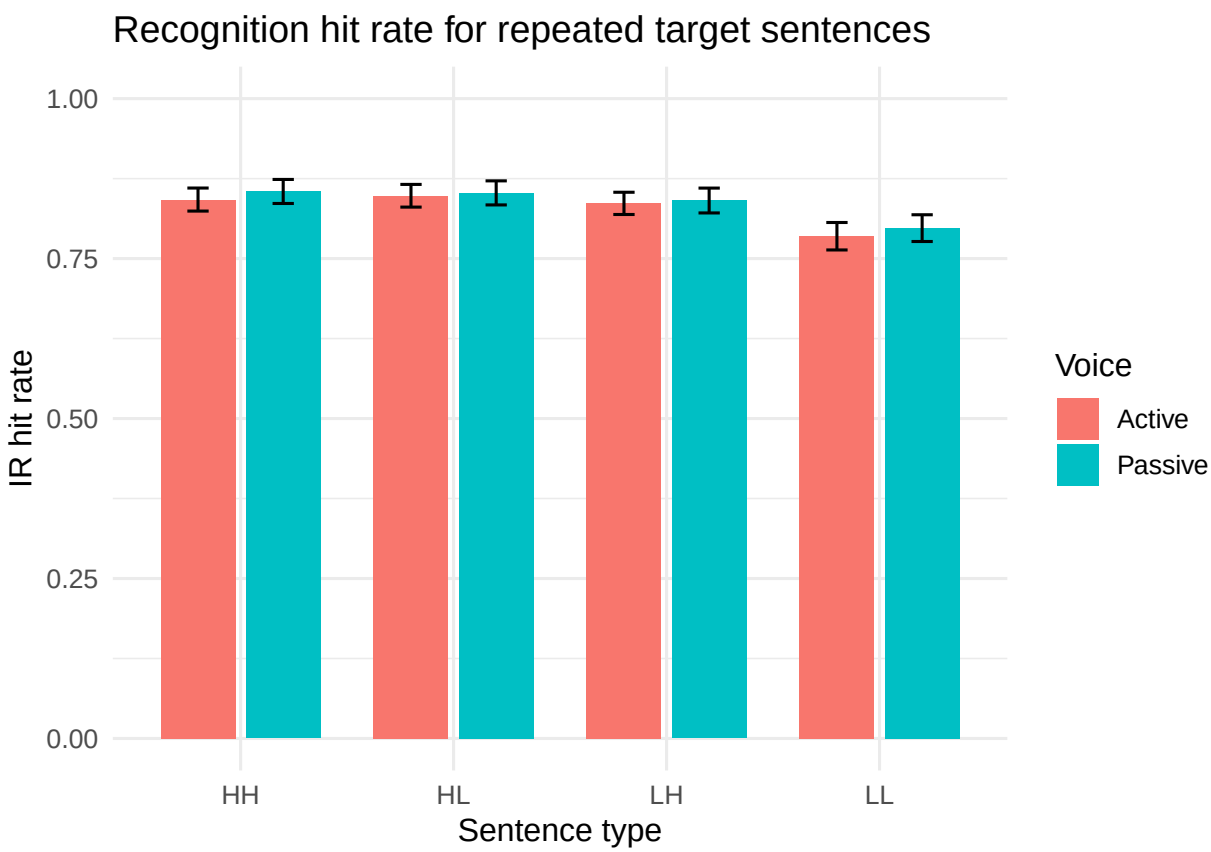
```
ggplot(score_plot_df,
       aes(x = interaction(stype, voice, sep = "-"),
           y = corrected_score, fill = voice)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  stat_summary(fun = median, geom = "point", size = 1.6, color = "black") +
  labs(
    title = "Distribution of corrected memorability scores",
    x = "Condition",
    y = "Corrected memorability score",
    fill = "Voice"
  ) +
  theme_minimal(base_size = 12)
```



Plot 4: raw hit rate for repeated targets

```
hit_plot_df <- hit_df %>%
  mutate(
    stype = factor(stype, levels = c("HH", "HL", "LH", "LL")),
    voice = factor(voice, levels = c("A", "P"), labels = c("Active", "Passive"))
  )

ggplot(hit_plot_df, aes(x = stype, y = hit_rate, fill = voice)) +
  stat_summary(fun = mean, geom = "bar",
               position = position_dodge(width = 0.8), width = 0.7) +
  stat_summary(
```

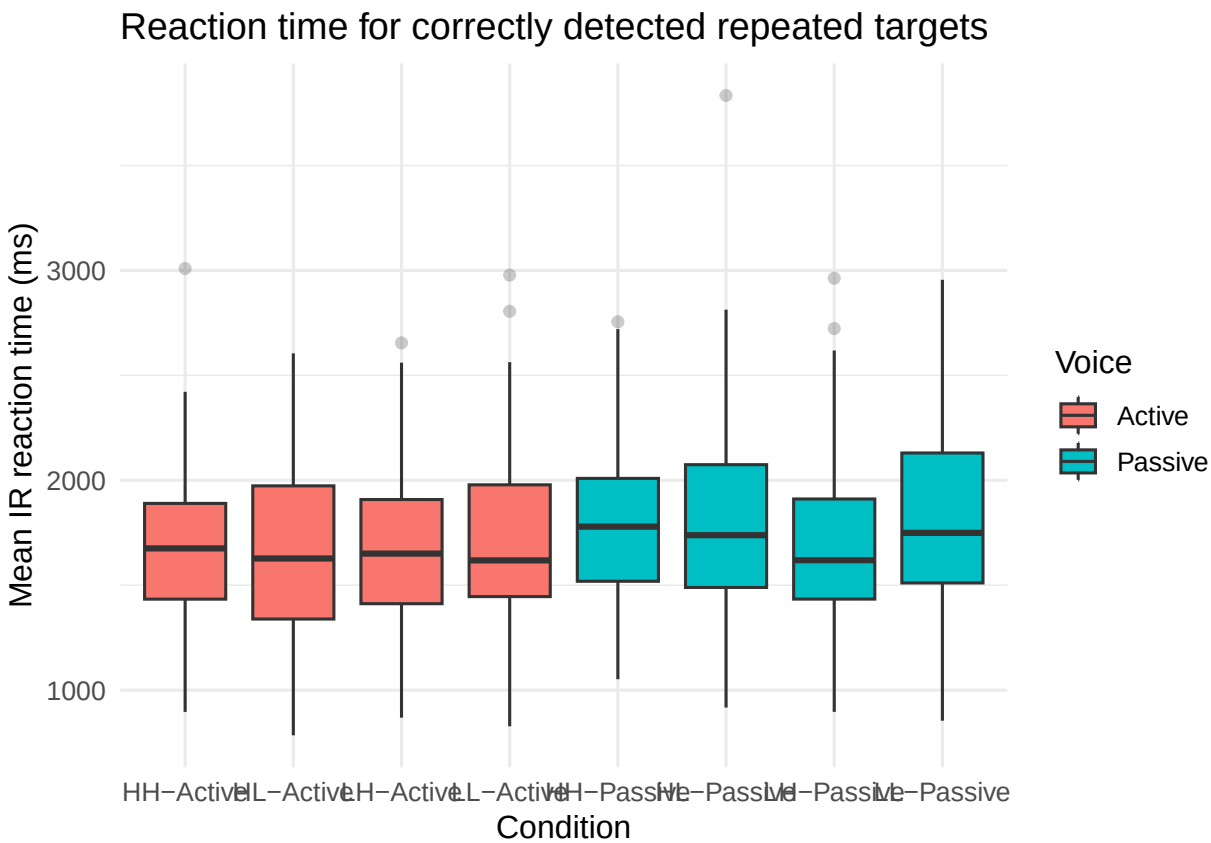
```
    fun.data = mean_se,
    geom = "errorbar",
    position = position_dodge(width = 0.8),
    width = 0.2
  ) +
  coord_cartesian(ylim = c(0, 1)) +
  labs(
    title = "Recognition hit rate for repeated target sentences",
    x = "Sentence type",
    y = "IR hit rate",
    fill = "Voice"
  ) +
  theme_minimal(base_size = 12)
```



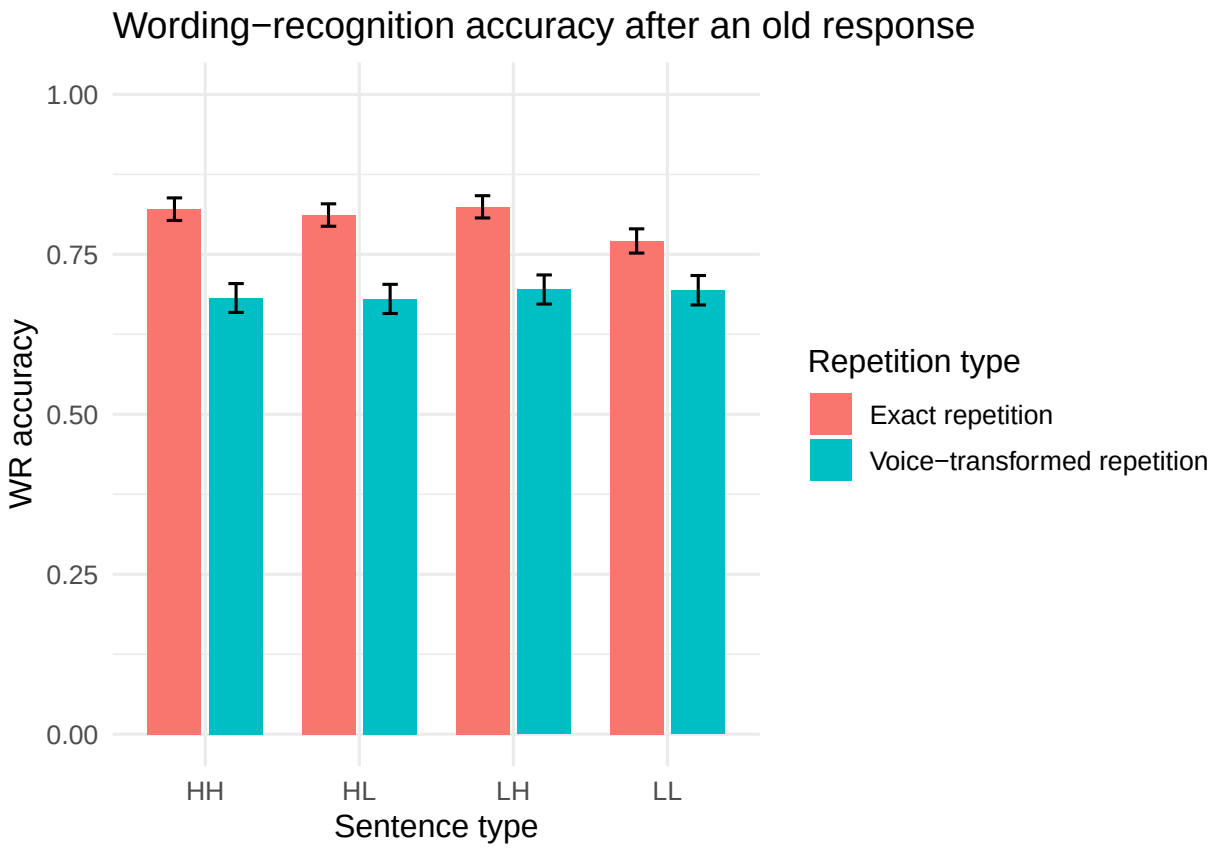
Plot 5: reaction-time spread

```
rt_plot_df <- rt_df %>%
  mutate(
    stype = factor(stype, levels = c("HH", "HL", "LH", "LL")),
    voice = factor(voice, levels = c("A", "P"), labels = c("Active", "Passive")),
    condition = interaction(stype, voice, sep = "-")
  )

ggplot(rt_plot_df, aes(x = condition, y = ir_rt_mean, fill = voice)) +
  geom_boxplot(outlier.alpha = 0.25) +
  labs(
```

```
    title = "Reaction time for correctly detected repeated targets",
    x = "Condition",
    y = "Mean IR reaction time (ms)",
    fill = "Voice"
  ) +
  theme_minimal(base_size = 12)
```



Plot 6: wording-recognition accuracy for exact vs similar repetitions

```
wr_plot_df <- wr_df %>%
  filter(rep_type %in% c("exact", "similar")) %>%
  mutate(
    stype = factor(stype, levels = c("HH", "HL", "LH", "LL")),
    rep_type = factor(
      rep_type,
      levels = c("exact", "similar"),
      labels = c("Exact repetition", "Voice-transformed repetition")
    )
  )

ggplot(wr_plot_df, aes(x = stype, y = wr_acc, fill = rep_type)) +
  stat_summary(fun = mean, geom = "bar",
               position = position_dodge(width = 0.8), width = 0.7) +
  stat_summary(
    fun.data = mean_se,
    geom = "errorbar",
```

```r
  position = position_dodge(width = 0.8),
  width = 0.2
) +
coord_cartesian(ylim = c(0, 1)) +
labs(
  title = "Wording-recognition accuracy after an old response",
  x = "Sentence type",
  y = "WR accuracy",
  fill = "Repetition type"
) +
theme_minimal(base_size = 12)
```